

Лабораторная №3. Использование кросс-компилятора

Цель работы

Научиться собирать на компьютере с архитектурой x86-64 программы для RISC-V, проверять их с помощью QEMU и запускать на целевой плате.

Оборудование и предварительные требования

- рабочий компьютер с ALT Linux на архитектуре x86-64;
- одноплатный компьютер Lichee RV Dock с архитектурой RISC-V;
- настроенное подключение к плате и средство передачи файлов;
- права суперпользователя на рабочем компьютере;
- редактор исходного кода.

Ключевые понятия

Определение

Кросс-компиляция — сборка программы на одной аппаратной или программной платформе для выполнения на другой целевой платформе.

Обычный цикл разработки для целевой архитектуры включает написание кода, сборку кросс-компилятором, перенос программы на плату или запуск в эмуляторе, а затем проверку и отладку.

Определение

Тулчейн (toolchain) — набор компиляторов, компоновщиков, библиотек и вспомогательных утилит для разработки под определённую платформу.

Основной компонент тулчейна в этой работе выполняется на x86-64, но создаёт программы для RISC-V. В ALT Linux нужные инструменты доступны в пакете `gcc-riscv64-linux-gnu` из репозитория для разработчиков.

Определение

QEMU user-mode — режим эмуляции, в котором программа для гостевой архитектуры выполняется на ядре хостовой Linux с трансляцией инструкций и системных вызовов.

Механизм `binfmt_misc` позволяет ядру Linux распознать исполняемый файл RISC-V и автоматически передать его зарегистрированному интерпретатору QEMU. Это даёт возможность проверить консольную программу на рабочем компьютере до переноса на плату.

Порядок выполнения

Настройка репозитория пакетов

Просмотрите текущую конфигурацию репозитория:

```
apt-repo
```

Необходимые инструменты доступны в репозитории Sisyphus. Если он ещё не настроен, сохраните текущий вывод `apt-repo`, затем переключите источники пакетов:

Опасность повреждения

Команда `apt-repo rm all` удаляет все подключённые репозитории из конфигурации APT. Выполняйте её только на предназначенной для лабораторной системе и заранее сохраните текущую конфигурацию, чтобы восстановить используемую ветку.

```
apt-repo rm all
apt-repo set sisyphus
apt-get update
```

Установка инструментов

Установите кросс-компилятор и поддержку запуска программ RISC-V через QEMU:

```
apt-get install gcc-riscv64-linux-gnu qemu-user-static-binfmt-riscv
```

Для написания программы используйте любой доступный редактор или IDE. В консольной среде можно работать с `vim`, `neovim`, `nano` или `emacs`.

Проверьте доступность компилятора:

```
riscv64-linux-gnu-gcc --version
```

Подготовка программы

Напишите на C приложение, которое принимает числовой аргумент из командной строки и с помощью математической библиотеки вычисляет пять значений выбранных элементарных функций. Предусмотрите проверку количества аргументов и корректности введённого значения.

Имя исходного файла в следующих командах замените на фактическое.

Кросс-компиляция

Соберите статически скомпонованный исполняемый файл и подключите математическую библиотеку:

```
riscv64-linux-gnu-gcc -static -o math-app math-app.c -lm
```

Внимание

Для показанного ниже запуска через QEMU нужна статическая компоновка. Без `-static` программе потребуются загрузчик и динамические библиотеки RISC-V, которых по умолчанию нет на рабочем компьютере.

Проверьте архитектуру полученного файла:

```
file math-app
```

Проверка

Ожидаемый результат. Утилита `file` должна определить исполняемый файл как 64-разрядный RISC-V ELF и указать статическую компоновку.

Запуск через QEMU

При настроенном `binfmt_misc` программу можно запустить как обычный исполняемый файл:

```
chmod +x math-app
./math-app аргумент
```

Если автоматический запуск не настроен, вызовите эмулятор явно:

```
qemu-riscv64-static ./math-app аргумент
```

Проверка

Проверка запуска. Программа должна вывести пять вычисленных значений и одинаково обрабатывать корректный аргумент при автоматическом и явном запуске через QEMU.

На рисунке показан пример работы приложения, запущенного через QEMU user-mode:

```
[taranev@taranev work_repos]$ ./test_cli 2
```

```
Math calculations for x = 2.0000:
```

```
=====
```

```
sqrt(x)    = 1.414214
x^2         = 4.000000
x^3         = 8.000000
sin(x)      = 0.909297
cos(x)      = -0.416147
```

Рис. 1 – Результат запуска приложения для RISC-V через QEMU user-static

Запуск на целевой плате

Передайте исполняемый файл на Lichee RV Dock с помощью `scp`, выдайте право на выполнение и запустите его с тем же аргументом. Сравните результат с запуском через QEMU.

Задание

1. Убедитесь в наличии или установите кросс-компилятор и подходящий редактор на рабочем компьютере.
2. Напишите приложение на C, которое принимает аргумент через интерфейс командной строки и с помощью библиотеки `libm` вычисляет пять значений выбранных элементарных математических функций.
3. Соберите приложение кросс-компилятором для RISC-V со статической компоновкой.
4. Проверьте архитектуру исполняемого файла.
5. Запустите программу через QEMU user-mode.
6. Передайте исполняемый файл на плату и выполните его.
7. Сравните результаты запуска на рабочем компьютере и плате.
8. Продемонстрируйте преподавателю исходный код, сборку и оба варианта запуска.

Контрольные вопросы

1. Что такое кросс-компиляция и в каких случаях она необходима?
2. Зачем нужен флаг `-static` при показанном запуске программы для RISC-V через QEMU?
3. Какую роль выполняют QEMU user-mode и `binfmt_misc`?
4. Чем различаются `gcc` и `riscv64-linux-gnu-gcc`?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- сведения об установленном кросс-компиляторе;
- исходный код приложения;
- команды и результаты кросс-компиляции;
- результат определения архитектуры файла утилитой `file`;
- результаты запуска через QEMU и на плате;
- сравнение полученных результатов;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.